NAME : PRERANA MUDIAR
ROLL NO : CSB24021

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// Structure for Dynamic String Buffer
typedef struct {
    char *data;
    size_t length;
    size_t capacity;
} StringBuffer;

// Function to initialize StringBuffer
StringBuffer* sb_init(size_t initial_capacity) {
    StringBuffer *sb = (StringBuffer*)malloc(sizeof(StringBuffer));

    if (sb == NULL) {
        printf("Memory allocation failed for StringBuffer\n");
        return NULL;
    }

    sb->data = (char*)malloc(initial_capacity * sizeof(char));

    if (sb->data == NULL) {
        printf("Memory allocation failed for data buffer\n");
        free(sb);
        return NULL;
    }

    sb->length = 0;
    sb->capacity = initial_capacity;

    // Empty string initialization
    sb->data[0] = '\0';

    return sb;
}

// Function to append string
void sb_append(StringBuffer *sb, const char *str) {
    if (sb == NULL || str == NULL)
```

```
        return;

    size_t str_len = strlen(str);

    // Check if resize is needed
    while (sb->length + str_len + 1 > sb->capacity) {

        size_t new_capacity = sb->capacity * 2;

        // Safe realloc
        char *temp = (char*)realloc(sb->data, new_capacity);

        if (temp == NULL) {
            printf("Reallocation failed\n");
            return;
        }

        sb->data = temp;
        sb->capacity = new_capacity;

        printf("Buffer resized! New Capacity = %zu\n", sb->capacity);
    }

    // Append string
    strcpy(sb->data + sb->length, str);

    sb->length += str_len;
}

// Destructor function
void sb_free(StringBuffer *sb) {
    if (sb != NULL) {
        free(sb->data);
        free(sb);
    }
}

// Main function
int main() {

    // Initial capacity = 8
    StringBuffer *sb = sb_init(8);
```

```c
    if (sb == NULL)
        return 1;

    printf("Initial Capacity = %zu\n", sb->capacity);

    // Append strings
    sb_append(sb, "Hello");
    printf("String: %s\n", sb->data);

    sb_append(sb, " World!");
    printf("String: %s\n", sb->data);

    sb_append(sb, " This is a dynamic string buffer in C.");
    printf("String: %s\n", sb->data);

    // Final details
    printf("\nFinal String = %s\n", sb->data);
    printf("Final Length = %zu\n", sb->length);
    printf("Final Capacity = %zu\n", sb->capacity);

    // Free memory
    sb_free(sb);

    printf("\nMemory freed successfully.\n");

    return 0;
}
```

OUTPUT:

```
[Running] cd "c:\Users\prerana\OneDrive\Desktop\PYTHON\" && gcc stringbuffer.c -o stringbuffer &&
"c:\Users\prerana\OneDrive\Desktop\PYTHON\"stringbuffer
Initial Capacity = 8
String: Hello
Buffer resized! New Capacity = 16
String: Hello World!
Buffer resized! New Capacity = 32
Buffer resized! New Capacity = 64
String: Hello World! This is a dynamic string buffer in C.

Final String = Hello World! This is a dynamic string buffer in C.
Final Length = 50
Final Capacity = 64

Memory freed successfully.

[Done] exited with code=0 in 0.303 seconds
```